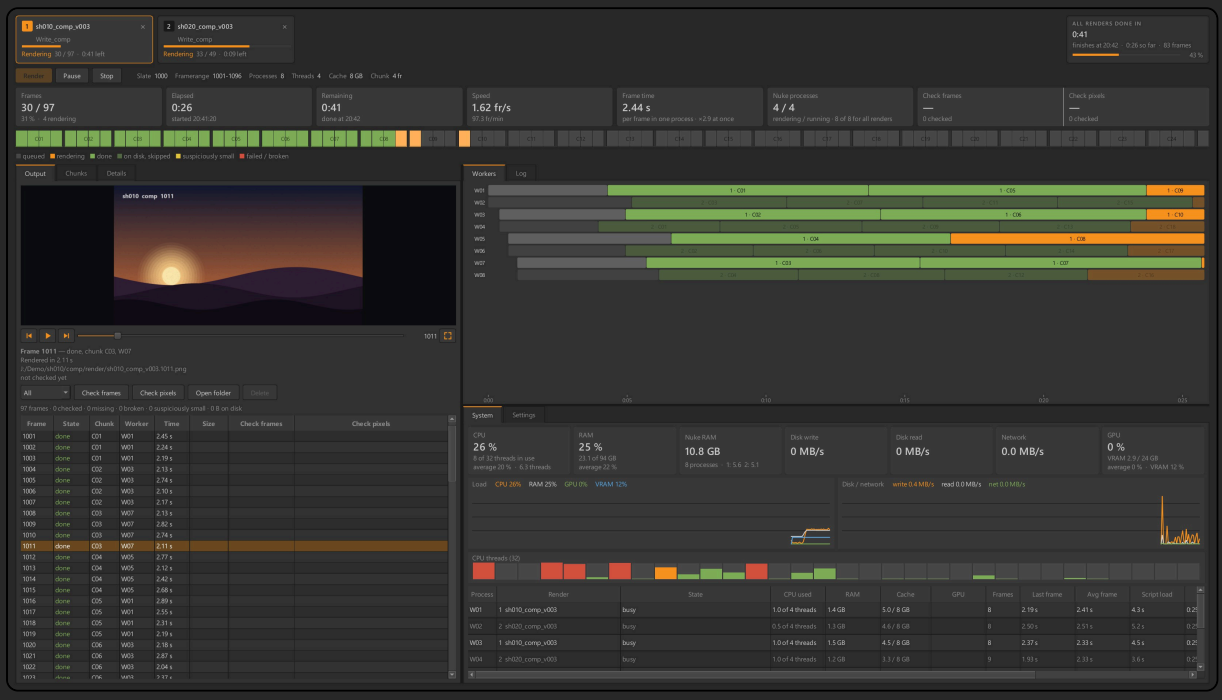
NÁVOD K POUŽITÍ · VERZE 0.94.3 ALPHA

JKrender

Rychlý lokální render pro Nuke. Render se rozdělí na kousky (chunky) a ty se počítají v několika Nuke procesech najednou. Po renderu JKrender zkontroluje každý frame na disku.

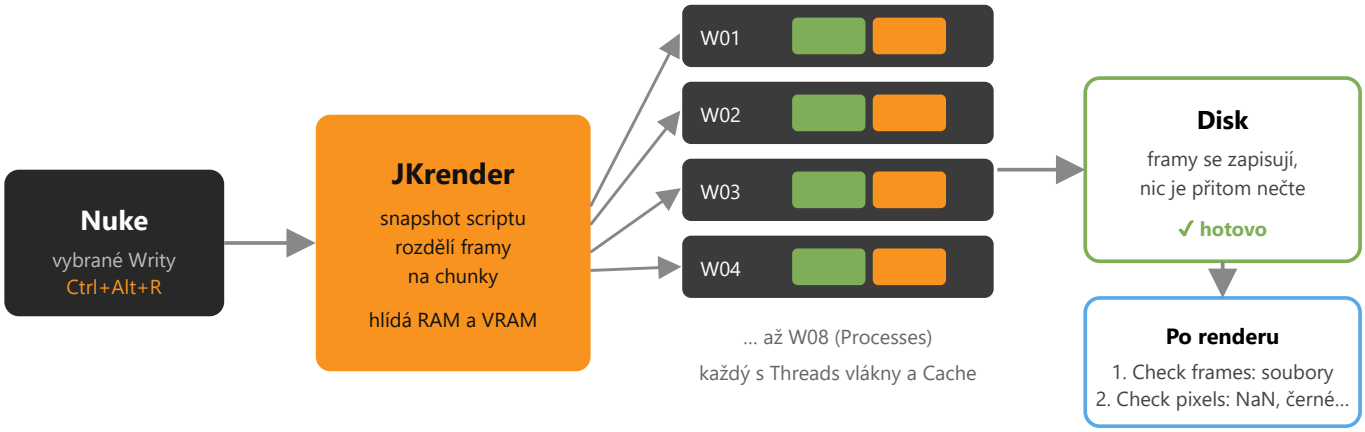


Obsah

1 Jak JKrender funguje
2 Instalace a spuštění z Nuku
3 Mapa okna
4 Karty renderů a ovládací řádek
5 Dlaždice a mapa framů
6 Panel Output: náhled a framy
7 Panely Chunks a Details
8 Panely Workers a Log
9 Panel System
10 Settings: každé nastavení
11 Jak nastavit render
12 Test this script
13 Hlídání RAM a VRAM
14 Kontroly, zvuky, pád, data
15 Myš a klávesy: tahák
16 Když něco nejde

1 Jak JKrender funguje

Klasický render v Nuku běží v jednom procesu, který frame po framu počítá celý script. Jeden Nuke ale málokdy vytíží všech 32 vláken procesoru. JKrender proto pustí **několik Nuke procesů vedle sebe**. Každý **otevře script jen jednou** a pak dostává kousky práce, takzvané **chunky**: třeba 10 framů za sebou.



Chunky se ke konci zmenšují, takže všechny procesy doběhnou zhruba zároveň.

Tři čísla, která rozhodují o rychlosti

Processes

Kolik Nuke procesů běží najednou (pro všechny rendery dohromady). Víc procesů znamená víc práce naráz, ale i víc RAM a VRAM.

Threads

Kolik vláken procesoru má každý proces (Nuke `-m`). **Processes × Threads** by mělo dát zhruba počet vláken PC (tady 32).

Cache

Limit cache jednoho procesu v GB (Nuke `-c`). Proces si ve skutečnosti vezme víc. Kolik, ukáže System › RAM.

> Výchozí nastavení je **8 procesů × 4 vlákna**. Těžké comp scripty, které jeden Nuke vytíží sám (Defocus, Denoise), bývají rychlejší na **4 × 8** a zaberou poloviční RAM. Scripty, kterým roste VRAM, chtějí méně procesů. Co je nejlepší pro tvůj script, změří **Test this script** (kapitola 12).

Co je kde uložené

Co	Kde
Nastavení	`~/.nuke/JKrender/settings/settings.json` (každý uživatel má svoje). Starší verze ho měly v `~/.jkrender`; JKrender ho při prvním spuštění sám přesune.
Logy renderů	`~/.nuke/JKrender/log/`: jedna složka na render (snapshot scriptu, job, log každého Nuke procesu a `report.json`). Starší než 15 dní se mažou.
Výsledky testů	`~/.nuke/JKrender/settings/test_results`
Vyrenderované framy	Tam, kam je posílá Write. JKrender je sám nikdy nemaže, jen přes tlačítko Delete.

2 Instalace a spuštění z Nuku

Instalace (jednou)

1. Zkopíruj celou složku `JKrender` do `%USERPROFILE%\.nuke\`. Když ji stáhneš z GitHubu jako zip, jmenuje se `JKrender-main`: přejmenuj ji na `JKrender`.

2. Do `~/.nuke/init.py` přidej řádek (soubor případně vytvoř):

```
nuke.pluginAddPath("./JKrender")
```

3. Restartuj Nuke. Při prvním startu se JKrender zeptá, jestli stáhnout numpy a OpenEXR pro náhled EXR (asi 15 MB). Když odmítneš, render funguje dál, jen bez náhledu EXR. Na dotaz se znovu dostaneš smazáním souboru `lib/DECLINED`.

Volitelně: pro detaily a přehrávač MOV/MP4 dej na PATH ffprobe/ffmpeg (`winget install ffmpeg`). Potřeba je Nuke 16 (PySide6).

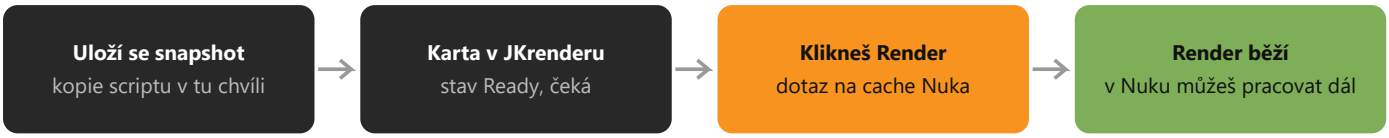
Nová verze: nakopíruj nové soubory přes staré. Složky `lib`, `log` a `settings` zůstanou.

Menu JKrender v Nuku

Položka	Co dělá
Render Selected Write(s) `Ctrl+Alt+R`	Pošle vybrané Writy (nebo Groupy s Writy) do JKrenderu. Každý Write dostane vlastní kartu. Když okno ještě neběží, otevře se.
Open Renders Folder	Otevře složku s logy renderů.

Když script není uložený, zeptá se *Save it and render?*: renderuje se uložená verze.

Co se stane po odeslání

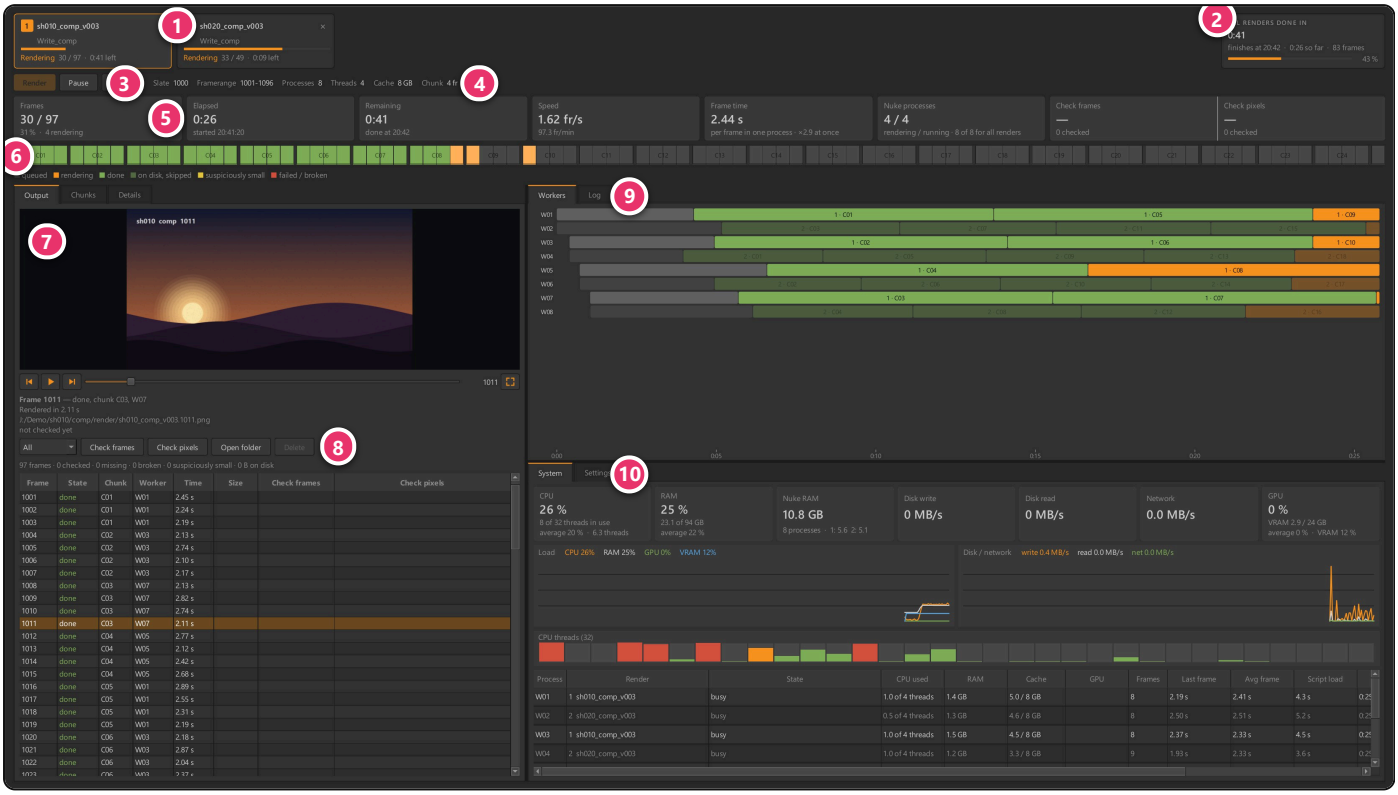


Snapshot: renderuje se script v podobě, jakou měl při odeslání. V Nuku můžeš hned dál pracovat a ukládat, na běžící render to nemá vliv. Když script změníš a chceš ho renderovat znovu, pošli ho z Nuku znovu: novější odeslání nahradí starou kartu.

Dotaz na cache: když je otevřený nějaký Nuke, Render se zeptá, jestli mu vyčistit buffery, playback cache a Blink cache, aby měl render víc paměti. Výchozí odpověď je **No** a render se spustí v obou případech. Diskovou cache to nemaže. Po aktualizaci JKrenderu je potřeba Nuke jednou restartovat, aby tohle fungovalo.

3 Mapa okna

Okno má nahoře karty renderů a ovládací řádek, pod nimi čísla a mapu framů. Zbytek tvoří tři panely: vlevo **Output / Chunks / Details**, vpravo nahoře **Workers / Log** a vpravo dole **System / Settings**. Hranice mezi panely jde táhnout myší.



1. Karty renderů: jeden Write (render) na kartu. Kliknutím ho vybereš.
2. Odhad, kdy budou hotové všechny rendery.
3. Render, Pause, Stop pro vybraný render.
4. Slate, Framerange, Processes, Threads, Cache a Chunk tohoto renderu (jen pro čtení).
5. Dlaždice s čísly: framy, čas, rychlost, procesy, kontroly.
6. Mapa framů po chuncích. Kliknutím na frame ho zobrazíš.
7. Output: náhled framu s přehráváním.
8. Output: filtr, kontroly, složka, mazání a seznam framů.
9. Workers: časová osa každého Nuke procesu.
10. System: zátěž PC a tabulka procesů.

4 Karty renderů a ovládací řádek

Karta renderu

Každý odeslaný Write je jedna karta: číslo, název scriptu, jméno Writu, pruh postupu a stav (Ready, Waiting, Rendering, Paused, Checking output, Done, Failed, Stopped). **Klik** kartu vybere a celé okno pak ukazuje tento render. **Shift+klik** vybere víc karet; Render, Pause a Stop pak platí pro všechny, Render again i pro hotové (zeptá se jednou za všechny). Křížek × render zavře, framy na disku zůstanou.

Rendery sdílejí jeden rozpočet procesů. Vedle sebe jich běží nejvýš tolik, kolik je v Settings *Max renders at once* (výchozí 3); procesy se mezi ně dělí (8 → 4 + 4 → 3 + 3 + 2).

Karta vpravo: ALL RENDERS DONE IN

Kdy by měly být hotové všechny rendery, které běží nebo čekají, při rychlosti, jakou PC renderuje teď. Pod tím: v kolik skončí, jak dlouho už běží a kolik framů zbývá. Když všechny doběhnou, karta ukáže **ALL RENDERS TOOK**, tedy jak dlouho to celé trvalo.

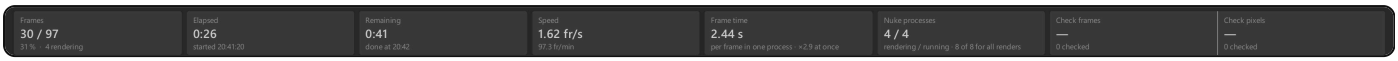
Ovládací řádek

Prvek	Co dělá
Render	Spustí vybraný render (nebo víc vybraných). Framy, které už jsou na disku v pořádku, přeskočí. Po Stopu render pokračuje tam, kde skončil (Resume). Když má render framy na disku (hotový, zastavený nebo s framy z dřívějška), zeptá se: **Only missing** dorenderuje jen framy, které na disku chybí nebo jsou rozbité (ostatní jsou v mapě tmavě zelené), **All frames again** vyrenderuje všechno znovu, **Cancel** nic.
Pause	Zmrazí všechny Nuke procesy renderu uprostřed framu (procesor se uvolní, paměť zůstane držená). Další klik pokračuje.
Stop	Zastaví render, ukončí procesy a smaže napůl zapsané framy a dočasné soubory Nuka. Hotové framy zůstanou.
Slate	Jestli se renderuje slate frame a na kterém framu (Settings › Job).
Framerange	Framy tohoto renderu: rozsah z Nuku, pokud jsi ho nezměnil v Settings › Job.
Processes · Threads · Cache · Chunk	S čím render běží. Mění se v Settings › Processes and Chunks; počet procesů i za běhu pravým tlačítkem v System.
Barevný pruh	Výsledek renderu, viz níže.

Barevný pruh s výsledkem

Barva	Co znamená	Příklad
Zelená	Hotovo, všechny framy jsou na disku a prošly kontrolou.	Done in 2:06. All 97 frames are on disk and pass the check.
Žlutá	Hotovo, ale něco není ověřené nebo vypadá podezřele (framy nezkontrolované, podezřele malé; slate se jako malý nehlásí).	frames not checked · 2 suspiciously small
Červená	Chyba: render selhal, rozbité framy, nebo Check pixels našel problém (NaN, černý frame…).	2 broken frames · Output: Problems shows them
Šedá	Probíhá: *Checking pixels…*, čekání na volné procesy a podobně.	Waiting: 1 render already uses the PC

5 Dlaždice a mapa framů



Dlaždice	Co ukazuje
Frames	Hotové / všechny framy renderu, procenta a kolik se jich právě renderuje.
Elapsed	Jak dlouho render běží (bez pauz).
Remaining	Odhad zbývajícího času při současné rychlosti.
Speed	Framy za sekundu a za minutu, všechny procesy dohromady. Nejlepší číslo pro porovnání nastavení.
Frame time	Kolik trvá jeden frame v jednom procesu a kolik framů se počítá najednou (× N at once).
Nuke processes	Kolik procesů renderuje / kolik jich běží, plus ty, které se načítají nebo čekají.
Check frames	Kontrola souborů: chybějící, prázdné, useknuté, nečitelné a podezřele malé framy.
Check pixels	Kontrola pixelů (EXR, PNG, JPEG, TIFF, DPX): černé, prázdné a jednobarevné framy, framy stejné jako jiný a frame, jehož velikost, kanály nebo bitová hloubka se liší od ostatních. U EXR navíc NaN, inf a komprese.

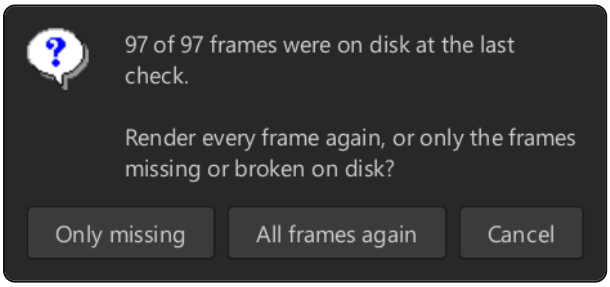
Mapa framů



Každý obdélníček je jeden frame, seskupené po chuncích (C01, C02…). **Klik na frame** ho otevře v náhledu. Barvy:

■ čeká (queued) ■ renderuje se ■ hotovo ■ byl už na disku, přeskočen (Only missing) ■ podezřele malý ■ selhal / rozbitý

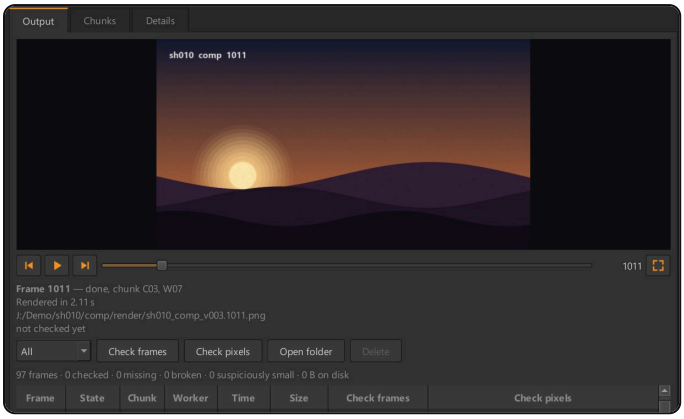
Render again: všechno, nebo jen chybějící



Když má render framy na disku a klikneš na **Render again**, zeptá se: **Only missing** (výchozí) dorenderuje jen framy, které chybí nebo jsou rozbité; ostatní přeskočí a v mapě jsou **tmavě zelené**. **All frames again** vyrenderuje všechno znovu, **Cancel** nic. U filmu (MOV, MP4) se nic neptá: film je jeden soubor a renderuje se celý.

S víc kartami vybranými přes **Shift** se zeptá jednou za všechny (např. *27 of 27 frames of 3 renders*…). Framy, které byly na disku už při otevření nebo z běhu před Stopem, jsou tmavě zelené. V Output pak filtr **Rendered** ukazuje jen framy z tohoto běhu, **All** všechny.

6 Panel Output: náhled a framy



Náhled

- **Kolečko myši**: zoom, **tažení** posouvá obraz.
- **Dvojklik**: obraz se vrátí na celý frame. Když už je celý, náhled se zvětší přes všechny tři panely; zpět klávesou `Esc`.
- Při zoomu se načte plné rozlišení (EXR i 4K).
- `Space` spustí a zastaví přehrávání. Nikdy nestiskne tlačítko, takže render nezastaví.

Lišta pod náhledem

Předchozí frame · **Play/Pause** (přehrává hotové framy dokola ve fps scriptu) · další frame · posuvník přes všechny framy · číslo framu · **zvětšit** (náhled přes tři panely, `Esc` zpět). Pod lištou jsou informace o framu: cesta, velikost, rozlišení, kanály a výsledky kontrol.

Tlačítka

Prvek	Co dělá
Filtr	**All**: všechny framy najednou: vyrenderované, přeskočené, rozbité, *not on disk* i ty, které se teprve budou renderovat, a framy na disku mimo Framerange. Ostatní filtry jsou jeho části: **Rendered**: jen framy vyrenderované v tomto běhu. **Problems**: chyby, podezřele malé framy a nálezy pixelové kontroly. **Framerange**: jen framy zvoleného rozsahu.
Check frames	Znovu projde soubory všech framů: existuje, není prázdný, je celý a čitelný a není výrazně menší než ostatní. Frame, který na disku ještě není, zůstane *not on disk*, za rozbitý se nepočítá.
Check pixels	Na pozadí projde pixely každého framu (EXR, PNG, JPEG, TIFF, DPX): černé, prázdné a jednobarevné framy, framy stejné jako jiný kdekoli v sekvenci, odlišná velikost, kanály nebo bitová hloubka; u EXR i NaN, inf a komprese. Zhruba 0,1 s na 4K frame. Jen hlásí, nic znovu nerenderuje.
Open folder	Otevře složku s výstupem.
Delete	Smaže vyrenderované framy z disku: všechny (výchozí) nebo zadaný rozsah. Předtím se zeptá.

Seznam framů

Sloupce: **Frame**, **State**, **Chunk**, **Worker** (který proces ho počítal), **Time**, **Size**, **Check frames**, **Check pixels**. S filtrem All jsou v seznamu od začátku všechny framy a stav se mění, jak se renderují. Po **Render again** framy nezmizí: zůstanou jako *on disk, queued again* se starou velikostí a časem (jdou prohlížet i přehrávat), dokud je nový render nepřepíše. Framy na disku mimo zvolený Framerange zůstávají v seznamu s poznámkou *not in Framerange*.

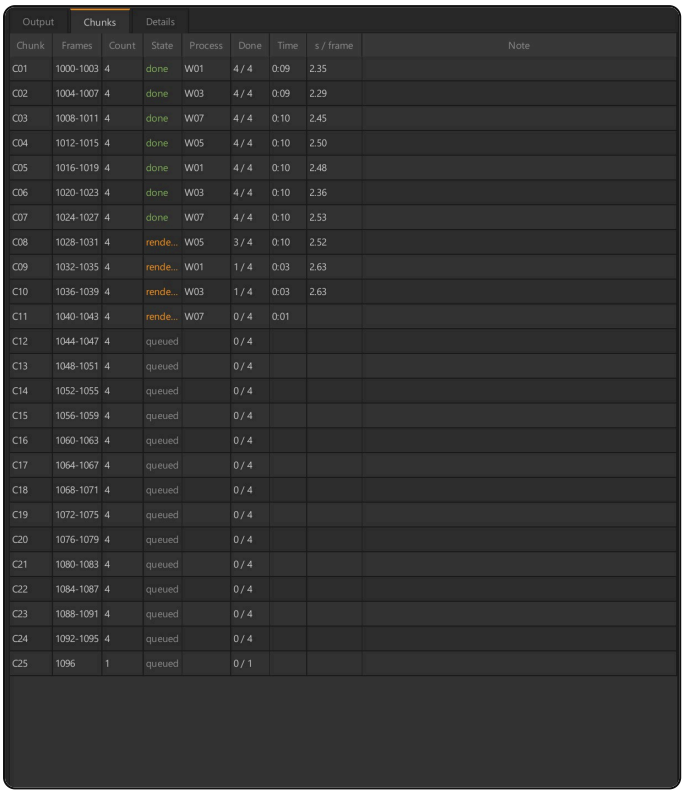


Pravé tlačítko na framu (nebo na víc vybraných):
Render this frame / batch: vyrenderuje jen tyto framy znovu (když render běží, přidají se do něj).
Delete this frame / batch: smaže je z disku (jen když render neběží).

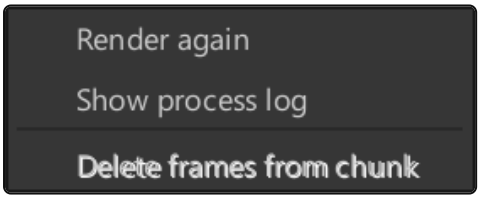
> **MOV / MP4 / MXF:** místo seznamu framů se ukáže jeden filmový soubor s detaily (kodek, profil, fps, rozlišení, timecode, počet framů proti nastavení Writu) a přehrávač s lupou. Tlačítka Check video, Open folder, Delete video. Film se nedá rozdělit, takže ho renderuje jeden proces se všemi vlákny; tak to ukazují i Settings (Processes 1, zamčené). Nastavení ostatních renderů se tím nemění.

7 Panely Chunks a Details

Chunks



Jeden řádek na chunk: **Chunk** (C01…), **Frames** (rozsah), **Count**, **State**, **Process** (který ho počítá), **Done**, **Time**, **s / frame** a **Note** (proč byl chunk zopakován a podobně).



Pravé tlačítko: Render again (chunk znovu), **Show process log** (log procesu, který ho počítal), **Delete frames from chunk** (jen když render neběží).

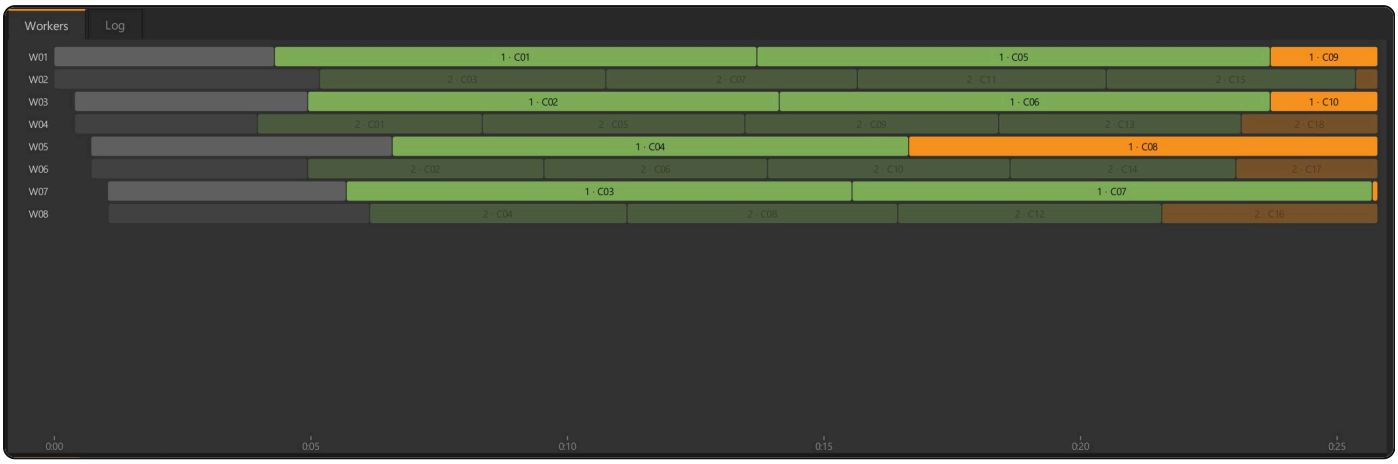
Details



Všechno o renderu na jednom místě: script a jeho snapshot, Writy a výstupní cesty, formát, fps, rozsah, verze Nuka, s jakými Processes × Threads × Cache běží, časy a cesty ke logům. Hodí se, když potřebuješ zjistit, co přesně se renderovalo.

8 Panely Workers a Log

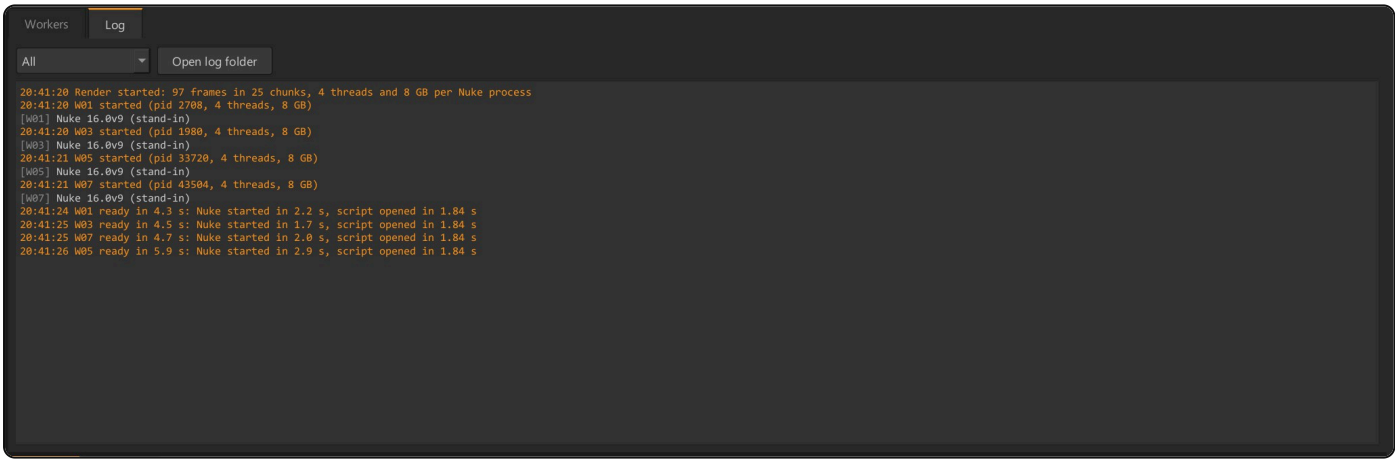
Workers: časová osa procesů



Jeden řádek na Nuke proces (W01–W08), zleva doprava čas. ⬛ **šedá**: Nuke startuje a otevírá script (i při výměně za čerstvý Nuke, popisek *again: fresh Nuke*). 🟩 **zelená**: hotové chunky (s číslem chunku). 🟧 **oranžová**: chunk, který se právě renderuje. Chunky ostatních renderů jsou ztlumené (Settings › *Dim other renders in Workers*). Je hned vidět, jestli některý proces stojí, startuje znovu nebo dlouho otevírá script.

Pravé tlačítko na řádku procesu otevře stejné menu jako v panelu System (kapitola 9).

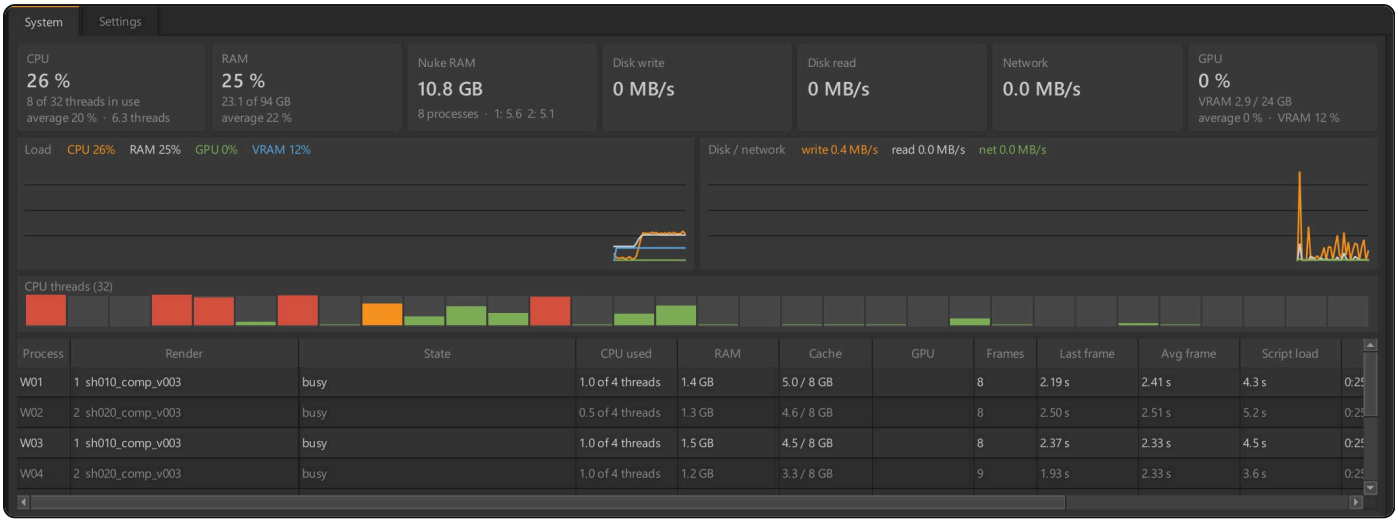
Log



Zprávy JKrenderu a výstup každého Nuke procesu s časem. Filtr: **All**, **JKrender only** (jen zprávy JKrenderu), **Errors and warnings**. Open log folder otevře složku s logy tohoto renderu: snapshot scriptu, `job.json` a `worker_NN.log` každého procesu.

> Každý proces v logu hlásí, za jak dlouho byl připravený, rozepsané na **start Nuka** a **otevření scriptu**. Pokud start trvá 15 s a víc, JKrender varuje. Pomalý start bývá licence nebo startovací skripty pipeline, ne script samotný.

9 Panel System



Dlaždice a grafy

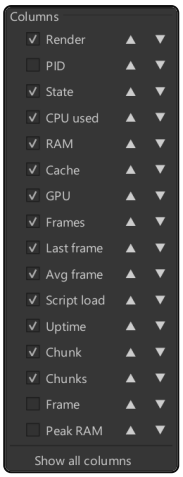
CPU (využití a vlákna), **RAM** (celková), **Nuke RAM** (kolik berou Nuke procesy), **Disk write**, **Disk read**, **Network**, **GPU** (využití a VRAM). Graf **Load** ukazuje CPU, RAM, GPU a VRAM v %, graf **Disk / network** přenosy v MB/s. **CPU threads**: jeden sloupeček na vlákno procesoru, takže je vidět, jestli je PC vytížené celé.

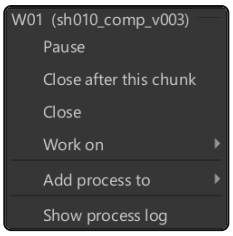
Tabulka procesů

Sloupec	Význam
Process	Číslo procesu (W01…).
Render	Na kterém renderu pracuje.
State	Co dělá: loading, busy, idle, paused, closing… *busy → fresh Nuke (RAM 91 %)*: po chunku dostane čerstvý Nuke, protože RAM je u limitu (kapitola 13). Při čekání na co, např. *wait: VRAM 75 → 14 % (3 starting)*: teď 75 %, další proces se vejde pod 14 %, protože 3 procesy se teprve rozjíždějí a JKrender jim drží místo; tooltip řekne víc.
CPU used	Kolik vláken procesoru proces opravdu vytěžuje.
RAM · Cache	Skutečná paměť procesu a jeho cache Nuka proti limitu.
GPU	Využití grafické karty a VRAM tohoto procesu.
Frames · Last frame · Avg frame	Kolik framů udělal, jak dlouho trval poslední a v průměru.
Script load	Za jak dlouho Nuke nastartoval a otevřel script.
Uptime · Chunk · Chunks	Jak dlouho běží, na jakém chunku je a kolik jich udělal.
PID · Frame · Peak RAM	Skryté sloupce: ID procesu ve Windows, aktuální frame, nejvyšší RAM.

Klik na hlavičku tabulky otevře seznam sloupců: zaškrtnutím sloupec ukážeš nebo skryješ, šipkami změníš pořadí. Sloupce jde přetahovat i myší za hlavičku. Šířky a pořadí se uloží při zavření okna.

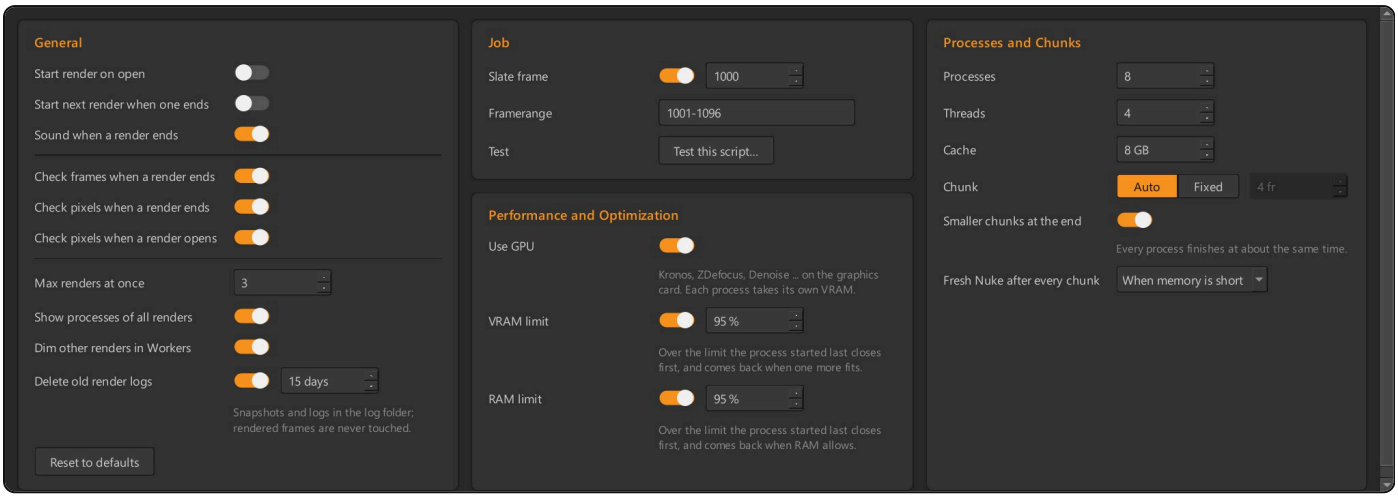
Pravé tlačítko na procesu (v tabulce nebo na řádku ve Workers) otevře menu vpravo. S víc běžícími rendery má *Add process to* podmenu s výběrem renderu.





Položka menu	Co dělá
Pause / Resume	Zastaví nebo pustí jen tento proces.
Close after this chunk	Proces dodělá svůj chunk a zavře se. **Processes** se sníží o 1 a zůstane tak.
Close	Zavře proces hned. Jeho chunk se vrátí do fronty. Na jeho místo pak může naskočit jiný.
Work on › render	Přesune proces na jiný běžící render.
Add process	Spustí ještě jeden proces (i kliknutím do prázdného místa tabulky). **Processes** se zvýší o 1 a zůstane tak. Limity RAM a VRAM platí dál.
Start again	U zavřeného procesu: spustí ho znovu.
Show process log	Otevře log tohoto procesu.

10 Settings: každé nastavení



Nastavení se ukládá hned. U každého je po najetí myší nápověda. **Reset to defaults** vrátí všechno na výchozí hodnoty včetně sloupců tabulek; co se JKrender naučil o scriptech, zůstane.

General

Nastavení	Co dělá	Výchozí
Start render on open	Render začne hned po otevření okna, bez kliknutí na Render.	vyp.
Start next render when one ends	Když render skončí nebo selže, sám se spustí další dosud nespuštěný render v pořadí karet. Render, který jsi zastavil, nic dalšího nespustí.	vyp.
Sound when a render ends	Zvuk po skončení renderu, až doběhnou zapnuté kontroly. Jiný zvuk při chybě, rozbitých framech nebo problému v pixelech. Zastavený render je potichu.	zap.
Check frames when a render ends	Po skončení renderu (i po Stopu) jednou projde každý vyrenderovaný frame: je tam, není prázdný, je celý a čitelný, není výrazně menší než ostatní. Během renderu výstupní složku nic nečte. Vypnuto: rozbitý frame se počítá jako hotový, dokud nestiskneš Check frames.	zap.
Check pixels when a render ends	Hned po kontrole framů na pozadí zkontroluje pixely všech framů (černé, jednobarevné, stejné, odlišný formát; u EXR i NaN a inf). Jen hlásí.	zap.
Check pixels when a render opens	Stejná kontrola framů, které už jsou na disku, hned po otevření renderu. Render ji zastaví.	zap.
Max renders at once	**1**: rendery jdou po sobě, další se začne načítat, když předchozí dodělává poslední chunky (nejdřív hotový první shot). **2–3**: běží vedle sebe a dělí se o procesy (celkově ne rychlejší, ale z každého shotu vidíš framy brzo).	3
Show processes of all renders	Workers a System ukazují procesy všech renderů. Vypnuto: jen vybraného.	zap.
Dim other renders in Workers	Na časové ose svítí chunky vybraného renderu, ostatní jsou ztlumené.	zap.
Delete old render logs	Při otevření JKrenderu smaže složky logů starší než nastavený počet dní. Framy nikdy.	zap., 15 dní

Skupiny v General oddělují tenké čáry. Při opakovaném renderu ukazuje časová osa Workers vždy jen nový běh.

Job (platí pro render vybraný nahoře)

Nastavení	Co dělá	Výchozí
Slate frame	Projekt začíná za slatem (např. 1001, slate na 1000). Zapnuto: každý render navíc vyrenderuje slate na tomto framu.	zap., 1000
Framerange	Framy renderu, např. `1001-1100` nebo `1001-1010,1050`. Každý render má svůj. Prázdné pole vrátí rozsah z Nuku. Potvrdíš Enterem nebo kliknutím jinam. Během renderu je zamčené. Framy na disku zůstávají.	z Nuku
Test › Test this script…	Změří nastavení podle tohoto PC na framech z celého shotu a ukáže nejrychlejší (kapitola 12).	—

Performance and Optimization

Nastavení	Co dělá	Výchozí
Use GPU	Spouští Nuke s `--gpu`, takže GPU nody (Kronos, ZDefocus, Denoise…) počítá grafická karta. Lehký 4K script s Kronosem byl 3× rychlejší. Každý proces má vlastní VRAM.	zap.
VRAM limit	Nad limitem běží méně procesů. Na 97 % se hned zavře proces spuštěný naposled; stejně tak, když je karta plná a žádný frame dlouho nedoběhne. Do 5 % od limitu pomáhá Fresh Nuke after every chunk. Kapitola 13.	zap., 95 %
RAM limit	Nad limitem běží méně procesů: první se zavře ten spuštěný naposled. Až se paměť uvolní, vrátí se. Windows tak nezačnou swapovat.	zap., 95 %

Processes and Chunks

Nastavení	Co dělá	Výchozí
Processes	Nuke procesy pro všechny rendery dohromady.	8
Threads	Vlákna na proces (`-m`).	4
Cache	Limit cache Nuka na proces v GB (`-c`). Kolik proces opravdu bere, ukazuje System › RAM. Příliš malá cache render zpomalí: Nuke pak zahazuje a přepočítává.	podle RAM
Chunk: Auto / Fixed	**Auto**: asi 4 chunky na proces, nejvýš 10 framů (46 framů s 8 procesy → 2, 96 → 3, 320+ → 10). **Fixed**: velikost zadáš vpravo. Platí pro chunky, které ještě nezačaly.	Auto
Smaller chunks at the end	Ke konci renderu se chunky zmenšují až na jednotlivé framy, takže všechny procesy doběhnou zhruba zároveň.	zap.
Fresh Nuke after every chunk	Proces po chunku zavře Nuke a otevře script v novém, s RAM a VRAM zase na začátku. Stojí to jedno načtení scriptu. **When memory is short**: jen když je RAM nebo VRAM 30 s v kuse do 5 % od limitu, a to po jednom procesu. **Always**: po každém chunku. **Off**: nikdy. Kapitola 13.	When memory is short

> Processes, Threads a Cache jsou během renderu zamčené. U filmu ukazují 1 proces se všemi vlákny. Počet procesů za běhu změníš pravým tlačítkem v System (*Add process*, *Close after this chunk*).

11 Jak nastavit render

Doporučený postup u nového nebo těžkého scriptu:

1	2	3	4	5
Pošli Write Ctrl+Alt+R	**Create tests** nastavení podle PC	**Start testu** Round 1 a Round 2	**Use these settings** převezme zelený řádek	**Render** a sleduj System

Kroky 2–4 jsou v okně Test this script. Lehké scripty klidně rovnou na výchozí 8 × 4; test se vyplatí, když render potrvá desítky minut.

Orientační nastavení (Ryzen 9950X, 32 vláken, 94 GB RAM, RTX 3090 24 GB)

Typ scriptu	Processes × Threads	Proč	Naměřeno
Lehký comp, keying, grade	**8 × 4**	Jeden Nuke nevytíží všechna vlákna, víc procesů ano.	testovací 4K script: 8 × 4 nejrychlejší
Těžký comp (Defocus, Denoise, hodně vrstev)	**4 × 8** nebo **2 × 16**	Jeden proces vytíží víc vláken. 8 procesů vyčerpá RAM, procesy se zavírají.	heavy comp: 4 × 8 s cache 12 GB 17 fr/min, 2 × 16 16 fr/min
Script, kterému roste VRAM (GPU nody)	**1–2 × 32/16**	VRAM roste s každým framem; víc procesů se na kartu nevejde.	VRAM +1,7 GB na frame → 1 × 32

Jak poznat, že nastavení není dobré

Málo paměti

System › RAM je u limitu, ve State je *wait: RAM …*, procesy se zavírají a znovu startují (šedé bloky ve Workers). **Řešení:** méně procesů s více vlákny, nebo menší Cache.

Plná grafická karta

VRAM u 95 %, procesy se zavírají kvůli VRAM. JKrender sám navrhne méně procesů s více vlákny (sám nic nemění). **Řešení:** méně procesů s více vlákny (Test this script ukáže kolik), případně vypnout Use GPU.

Nevyužité CPU

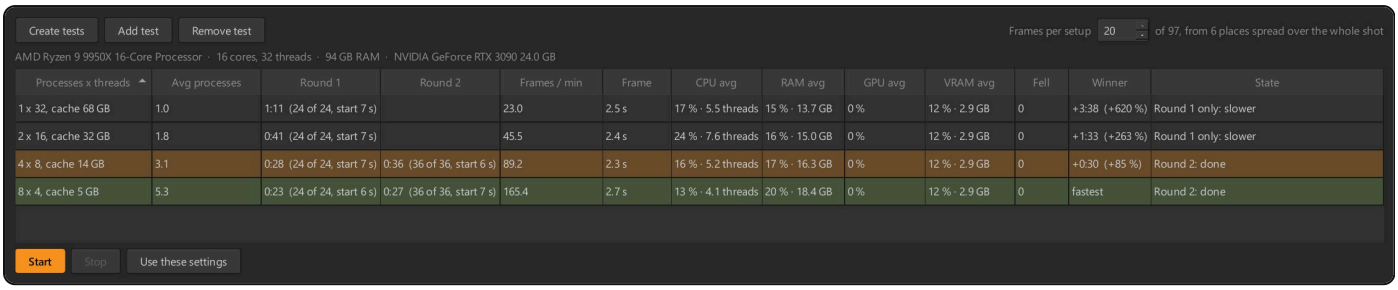
CPU threads jsou z velké části prázdné a RAM má rezervu. **Řešení:** přidat proces (pravé tlačítko › Add process) nebo zvýšit Threads.

Dlouhé otevírání scriptu

Šedé bloky ve Workers jsou dlouhé, Script load je vysoký. Při startu se Nuky přetahují o licence a startovací skripty. **Řešení:** delší chunky se tolik neprojeví; se samotným scriptem to obvykle nesouvisí.

> **Nejlepší měřítko je dlaždice Speed** (framy za minutu). Porovnávej ji mezi nastaveními na stejném scriptu, nebo to nech na *Test this script*.

12 Test this script



Hotový test: zelený řádek je nejrychlejší, oranžový druhý. Winner ukazuje, o kolik déle by celý shot trval s ostatními.

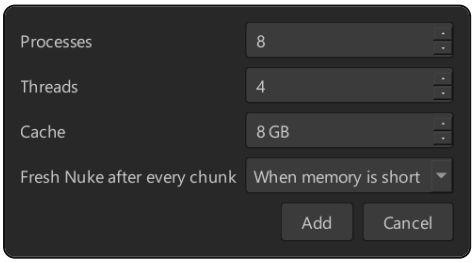
Tlačítka nahoře

Create tests zjistí, jaký je počítač (řádek pod tlačítky: CPU, jádra a vlákna, u Intelu výkonná a úsporná jádra, RAM, grafika) a vytvoří **7 nastavení** od 1 procesu se všemi vlákny po tolik procesů, kolik PC unese: každý proces aspoň 4 vlákna a aspoň 4 GB cache. Na 32 vláknech a 94 GB: 1×32, 2×16, 3×10, 4×8, 5×6, 6×5, 8×4. Nad 64 vlákny dostane proces nejvýš 64 (jedna skupina procesoru ve Windows). Nahradí vše v tabulce.

Add test přidá vlastní nastavení (Processes, Threads, Cache a Fresh Nuke after every chunk), Remove test odebere vybrané. **Pravé tlačítko** na řádku: **Edit test**. Vlastní testy se pamatují. Tabulka ukazuje Fresh Nuke v názvu nastavení: *4 x 8, cache 12 GB, fresh every chunk* nebo *never fresh*; bez dovětku je to When memory is short.

Cache každého nastavení je celá RAM pro Nuke (RAM limit, bez rezervy pro Windows) dělená počtem procesů, minus 4 GB, které proces bere mimo cache. Nezáleží na tom, co zrovna běží.

Frames per setup (vpravo nahoře): kolik framů vyrenderuje každé nastavení v Round 2. Když má shot méně framů, ukáže to a renderují se všechny.



Jak test probíhá

Framy se berou z **6 míst rozprostřených po celém shotu** (od prvního po poslední frame), na každém pár framů za sebou. **Všechna nastavení v kole renderují stejné framy.**

Round 1: každé nastavení krátce, 3 framy na proces nastavení s nejvíc procesy (na 32 vláknech 24 framů), nejvýš 3 minuty. **Round 2:** dvě nejrychlejší (a každé do 5 % od nejlepšího) znovu s Frames per setup, aspoň 4 framy na proces.

Frames / min se počítá, až procesy jedou: start Nuka a první frame každého procesu se nepočítají, v dlouhém renderu jsou jen jednou.

Sloupce: **Avg processes** (kolik procesů v průměru renderovalo), **Round 1**, **Round 2** (čas kola, framy, start), **Frame**, **CPU / RAM / GPU / VRAM avg**, **Fell** (spadlé procesy), **Winner**, **State**. Klik na záhlaví řadí, sloupce jdou roztáhnout a přesunout.

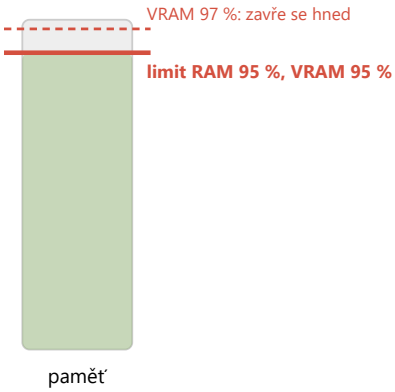
Use these settings převezme zelený řádek do Settings, i s Fresh Nuke. Framy jdou do testovací složky a mažou se; render hooky Writu se v testu nespouští. Tabulky zůstávají v `~/.nuke/JKrender/settings/test_results`.

Okénko **Add test** (a stejné **Edit test** z pravého tlačítka).

> Během testu by nemělo nic jiného renderovat: test by měřil dva rendery, které se perou o PC. Test běžící vedle renderu nepustí.

13 Hlídání RAM a VRAM

JKrender hlídá paměť, aby se Windows nezačaly swapovat a grafická karta se nezaplnila.



Do 5 % od limitu 30 s v kuse
Procesy po chunku postupně, po jednom, dostanou čerstvý Nuke (Fresh Nuke).

Nad limitem
Zavře se proces spuštěný naposled: když ještě otevírá script hned, jinak po chunku.

Návrat
Vrátí se najednou tolik procesů, kolik se vejde; startují postupně.

Opakované přetížení
Čeká se 5 s, pak 30, 60 a 150 s, když se to opakuje do 10 minut.

Když jde proces zpátky, musí se pod limit vejít jeden proces té velikosti, jakou v tomto renderu procesy měly (do té doby jeho Cache + 4 GB), s rezervou 2 % a s tím, co běžící procesy ještě naberou. Na co se právě čeká, ukazuje sloupec **State** v System, např. *wait: VRAM 75 → 69 %*: VRAM je na 75 % a musí klesnout pod 69 %, aby se vešel další proces. *wait: VRAM 75 → 14 % (3 starting)*: tři procesy se teprve rozjíždějí a JKrender jim drží místo v plné velikosti. Další stavy: *wait: 25 s* (proces právě spadl), *wait: W03 1st frame* (čeká na první frame W03, aby bylo vidět, kolik proces bere), *wait: VRAM ok, checking* (místo je, ověřuje se, že vydrží).

Plná karta a nic nejede: když je VRAM nad limitem a žádný proces nedokončí frame déle než 4× obvyklou dobu framu (aspoň minutu), Nuke nejspíš přešel na CPU a všechny procesy čekají. Naposled spuštěný proces se pak zavře hned a jeho chunk se vrátí do fronty.

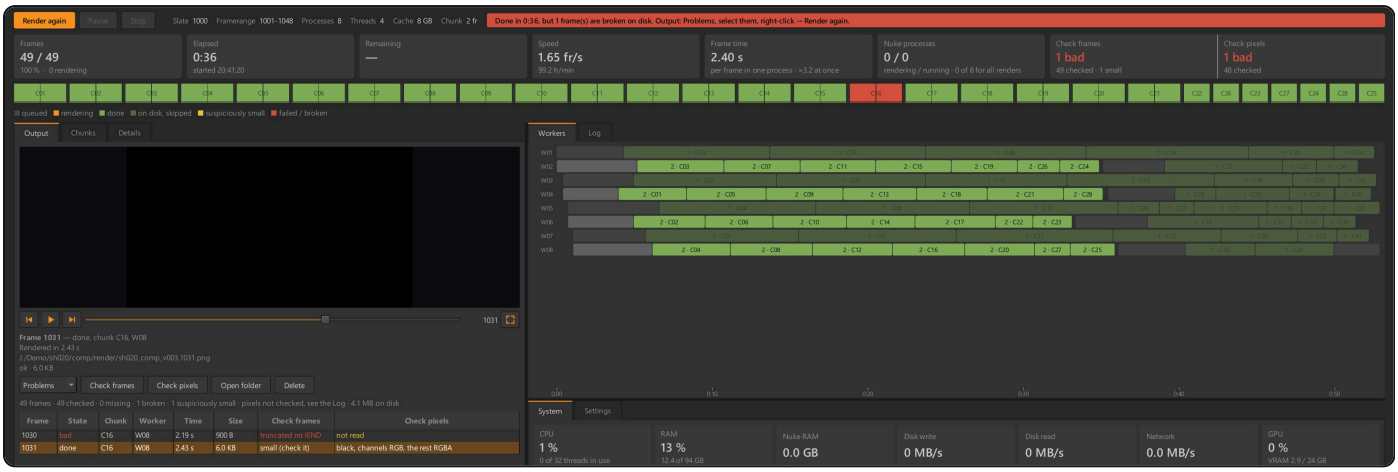
Fresh Nuke after every chunk

Nuke si při renderu drží paměť, kterou už nepotřebuje: proces těžkého scriptu může během prvních ~20 framů narůst ze 2 na 7 GB VRAM. Čerstvý Nuke začne zase malý, takže se na kartu vejde víc procesů. Ve výchozím **When memory is short** se to stane, jen když je RAM nebo VRAM **30 s v kuse** do 5 % od limitu (jedno přeskočení nestačí), a **po jednom procesu**: další až když nový Nuke vyrenderuje první frame a paměť je pořád u limitu. Ostatní mezitím renderují dál. Proces, který dostane čerstvý Nuke, ukazuje ve State *busy → fresh Nuke (VRAM 93 %)*; u posledních chunků se nic nevyměňuje.

VRAM strop na proces neexistuje. Cache (`-c`) omezuje jen RAM cache Nuka. Paměť grafické karty jeden proces omezit nejde: Windows to neumí a nastavení Blink cache v Nuku se v renderu neprojeví (změřeno). Proto JKrender řídí VRAM počtem procesů a čerstvými Nuky. Cache je limit cache, ne celková paměť procesu; skutečné číslo je v System › RAM.

14 Kontroly, zvuky, pád a data

Kontrola výstupu



Render s useknutým framem (1030) a černým framem (1031), oba najdené ručním Check frames a Check pixels: červený pruh, červené framy v mapě, filtr Problems ukazuje jen je.

Kontrola	Kdy	Co hledá
Check frames	Po skončení renderu a po Stopu (lze vypnout) a tlačítkem. Vždy při otevření renderu a před Renderem.	Chybějící, prázdné, useknuté a nečitelné soubory (EXR, PNG, JPEG, DPX), podezřele malé framy.
Check pixels	Po skončení renderu, při otevření (lze vypnout) a tlačítkem.	EXR, PNG, JPEG, TIFF a DPX: černé, prázdné a jednobarevné framy, framy stejné jako jiný, odlišná velikost, kanály a bitová hloubka. U EXR navíc NaN, inf a komprese. Ostatní formáty: jen stejné soubory.
Movie check	U MOV, MP4 a MXF.	Kodek, profil, fps, rozlišení, timecode a počet framů proti nastavení Writu.

Zvuk po renderu

Po renderu běží kontroly v pořadí **Check frames** → **Check pixels**; zvuk zazní, až doběhnou ty zapnuté (s vypnutými hned po renderu).

Veselý zvuk: všechno v pořádku. **Chybový zvuk**: render selhal, má rozbité framy nebo pixelová kontrola našla problém. Ručně zastavený render je potichu.

Když JKrender spadne

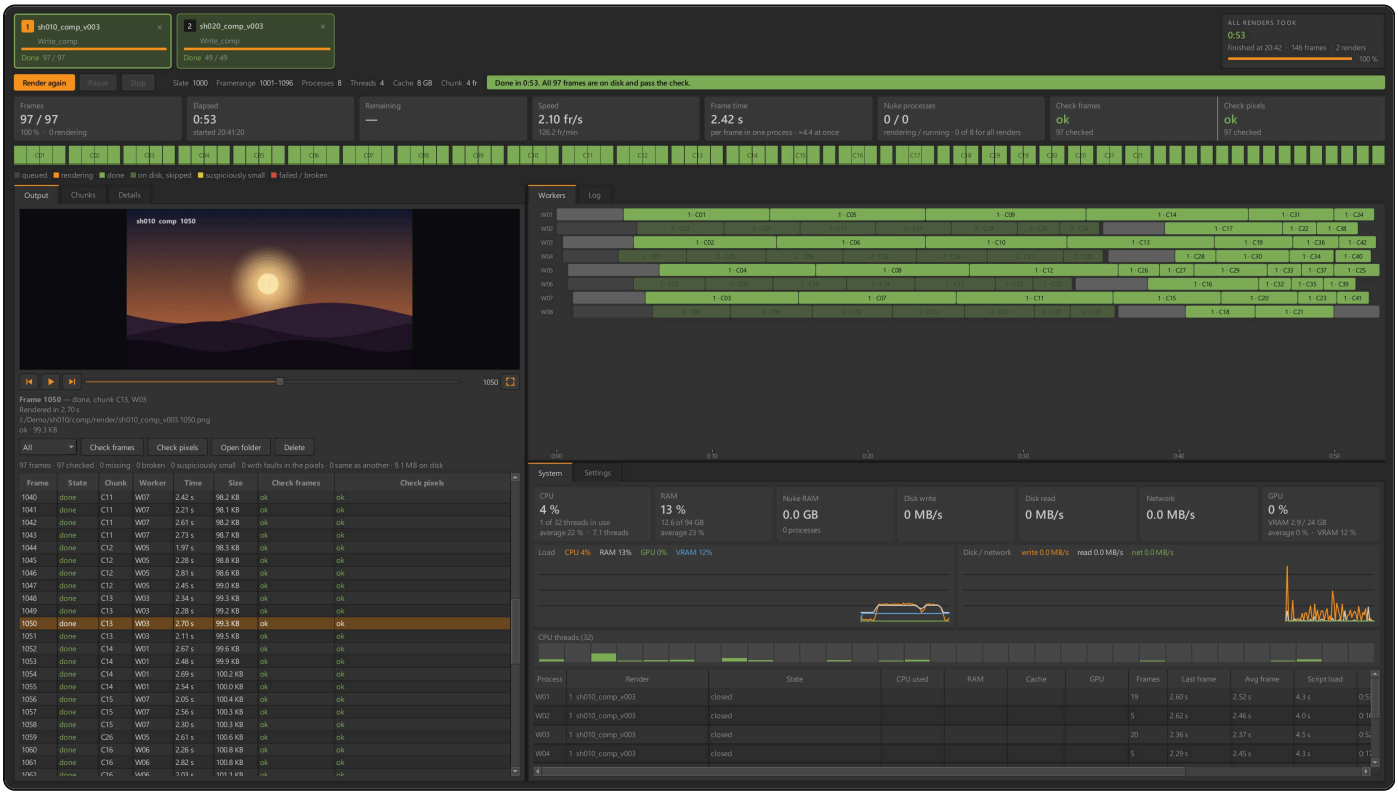
Karty se vrátí při dalším otevření JKrenderu (třeba dalším renderem poslaným z Nuku). Co je na disku, se započítá a zbylé Nuke procesy se ukončí. Render, který běžel, čeká na Render a pak dorenderuje, co chybí. Když okno zavřeš normálně, nic se nevrací.

Render data

Každý render zapíše do své složky logu `report.json`: CPU, RAM a grafickou kartu tohoto PC, nastavení, rychlost, špičky paměti, zavírání kvůli limitům, start Nuka, otevření scriptu a druhy nodů ve scriptu. **Nejsou v něm názvy scriptů, cesty ani chyby**; script se pozná jen podle ID, které je stejné napříč jeho verzemi. Když narazíš na chybu, pošli složku logu renderu.

15 Myš a klávesy: tahák

Kde a co	Výsledek
`Ctrl+Alt+R` v Nuku	Pošle vybrané Writy do JKrenderu.
Klik na kartu renderu	Vybere render; celé okno ukazuje jeho data.
`Shift` + klik na karty	Vybere víc renderů pro Render, Render again, Pause, Stop.
Klik na frame v mapě framů	Otevře frame v náhledu.
`Space`	Přehrát / zastavit náhled (nikdy nezastaví render).
Kolečko / tažení v náhledu	Zoom / posun obrazu.
Dvojklik v náhledu	Celý frame; když už je celý, velký náhled.
`Esc`	Zpět z velkého náhledu.
Pravé tlačítko na framu (Output)	Render this frame / batch, Delete this frame / batch.
Pravé tlačítko na chunku (Chunks)	Render again, Show process log, Delete frames from chunk.
Pravé tlačítko na procesu (System, Workers)	Pause, Close after this chunk, Close, Work on, Add process, Start again, Show process log.
Pravé tlačítko do prázdného místa v System	Add process.
Klik na hlavičku tabulky System	Výběr a pořadí sloupců.
Tažení hranice panelů	Změní velikost panelů (uloží se).
Framerange + `Enter`	Potvrdí nový rozsah (Settings › Job).



Hotový render: zelený pruh, karta vpravo nahoře ukazuje, jak dlouho rendery trvaly.

16 Když něco nejde

Problém	Co s tím
Nuke procesy dlouho startují (30–50 s)	Obvykle start Nuka (licence, startovací skripty pipeline) ve chvíli, kdy PC renderuje. Log ukazuje rozdělení *Nuke started in …* / *script opened in …*.
Procesy se zavírají a startují znovu	Došla RAM nebo VRAM (State v System, Log). Méně procesů s více vlákny nebo menší Cache; Test this script ukáže, co je nejrychlejší.
Ve State je *busy → fresh Nuke*	RAM nebo VRAM je 30 s u limitu; proces po chunku dostane čerstvý Nuke (Settings › Fresh Nuke after every chunk). Když se to děje pořád, běží procesů víc, než se vejde.
Render je celý pomalejší než minule	Zkontroluj Cache: příliš malá (např. 4 GB u těžkého scriptu) znamená, že Nuke zahazuje a přepočítává, a framy trvají několikrát déle.
Script se v Nuku otevírá s chybou knobu	Např. *LensDistortion2.serialiseKnob: Failed to load* (node z jiné verze, chybějící plugin). JKrender renderuje dál, jako to dělá Nuke v GUI; hláška je v logu procesu.
Žlutý pruh „not checked“	Kontrola framů je vypnutá (Check frames when a render ends) nebo neproběhla. Stiskni Check frames.
Červený pruh	Output › filtr Problems ukáže, které framy. Pravým tlačítkem *Render this frame / batch*. Pruh se přepočítá i po ručním Check frames a Check pixels.
Náhled EXR se neukazuje	Chybí numpy/OpenEXR. Smaž `lib/DECLINED` a restartuj Nuke, nebo je nainstaluj podle README.
Render nezačal hned	Stav Waiting: jiné rendery už vytěžují PC (Max renders at once), nebo se čeká na paměť.
Vyčištění cache Nuka nefunguje	Po aktualizaci JKrenderu je potřeba Nuke jednou restartovat.
Nevím, co se renderovalo	Panel Details a Log › Open log folder (snapshot scriptu a job.json).

> **JKrender je v alpha verzi.** V testech funguje, ale počítej s drobnostmi. Když narazíš na chybu, pošli autorovi složku logu renderu (Open Renders Folder) a popis, co se stalo.

JKrender 0.94.3 alpha · github.com/JanKanta/JKrender · návod aktualizován 9. 11. 2026. Snímky jsou z ukázkového renderu (shoty sh010 a sh020).